

AI ASSISTED CODING PROJECT

Team members:

J.Rohith – 2403A510F1

H.V.S.Vyshnavi – 2403A51182

J.Abhiram – 2403A51342

Traces and Logs

Objective: Add structured logging and request tracing to an app using AI to recommend fields and instrumentation points.

Steps: Add logger, instrument middleware, sample traces, and dashboard README.

1.ABSTRACT

In modern distributed systems, understanding system behavior and diagnosing issues requires detailed observability. This project integrates structured logging and request tracing into a sample web application. Using AI-assisted recommendations, the system suggests optimal log fields and instrumentation points to enhance traceability, debugging, and performance monitoring. The approach ensures improved visibility across services, facilitating faster incident resolution and proactive optimization. The final implementation includes middleware-based instrumentation, sample log traces, and a visualization dashboard for insights.

This project demonstrates how to add structured logging and distributed request tracing to a simple web application. We build a minimal Python Flask app and instrument it with:

- Structured logging using structlog (with the stdlib logging integration) so logs are JSON-friendly and contain trace context.
- Distributed tracing using OpenTelemetry (OTel) with an OTLP exporter (console/collector) and automatic/instrumented middleware. We provide step-by-step instructions, code, sample outputs, a flow chart, and a dashboard README to visualize logs and traces (example using Jaeger/Tempo + Grafana or OpenTelemetry Collector).

OBJECTIVES

1. Implement structured logging using a consistent, machine-readable format (JSON).
2. Add request tracing to follow a request's lifecycle across the application.
3. Use AI to recommend:
 - Critical log fields (e.g., request_id, user_id, latency, status_code).

- Instrumentation points (middleware, database calls, external APIs).
4. Build a dashboard README explaining how to visualize logs and traces.
 5. Provide sample trace outputs for demonstration.

TOOLS AND TECHNOLOGIES

Category	Tools / Libraries
Programming Language	Python 3
Framework	Flask (for web app)
Logging	structlog, logging
Tracing	OpenTelemetry, opentelemetry-instrumentation-flask
Exporter	Jaeger / Zipkin
AI Recommendation	OpenAI GPT model (for recommending fields/instrumentation points)
Visualization	Jaeger / Grafana
Deployment	Docker / Localhost

CODE:

```
import json
import time
import uuid
from http.server import BaseHTTPRequestHandler, HTTPServer

# =====
# AI-Simulated Recommendations (Fields & Instrumentation)
# =====

AI_RECOMMENDATIONS = {
    "recommended_fields": [
        "timestamp", "service_name", "trace_id", "span_id",
        "method", "path", "status_code", "latency_ms", "client_ip"
    ],
    "instrumentation_points": [
        "request_start", "request_end", "error_handler", "business_logic"
    ]
}

# =====
# STRUCTURED LOGGER
# =====

def log_event(event, **fields):
    """Structured JSON logging."""
    log_entry = {
        "timestamp": time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime()),
        "event": event,
        "fields": fields
    }
```

```

    print(json.dumps(log_entry, indent=2))

# =====
# TRACE MANAGEMENT
# =====

def start_trace():
    """Start a simulated trace (trace_id + span_id)."""
    trace_id = str(uuid.uuid4())
    span_id = str(uuid.uuid4())[:8]
    log_event("trace_started", trace_id=trace_id, span_id=span_id)
    return trace_id, span_id

def end_trace(trace_id, span_id, latency_ms):
    """End a simulated trace."""
    log_event("trace_ended", trace_id=trace_id, span_id=span_id, latency_ms=latency_ms)

# =====
# SIMPLE WEB SERVER
# =====

class SimpleHandler(BaseHTTPRequestHandler):
    service_name = "structured-logger-demo"

    def do_GET(self):
        start_time = time.time()
        trace_id, span_id = start_trace()
        client_ip = self.client_address[0]

```

```

# Simulate middleware instrumentation (AI recommended)
log_event("request_start",
    service_name=self.service_name,
    method=self.command,
    path=self.path,
    client_ip=client_ip,
    trace_id=trace_id,
    span_id=span_id)

# Route handling
if self.path == "/":
    response = {"message": "Hello from Structured Logging & Tracing Demo!"}
    log_event("business_logic", action="home_accessed")
    status_code = 200
elif self.path == "/compute":
    result = sum(range(100000))
    response = {"result": result}
    log_event("business_logic", action="compute_executed", result=result)
    status_code = 200
elif self.path == "/error":
    try:
        1 / 0
    except Exception as e:
        response = {"error": str(e)}
        log_event("error_handler", error=str(e))
        status_code = 500
else:
    response = {"error": "Not Found"}

```

```

        log_event("business_logic", action="not_found", path=self.path)
        status_code = 404

    # Send response
    self.send_response(status_code)
    self.send_header("Content-Type", "application/json")
    self.end_headers()
    self.wfile.write(json.dumps(response).encode())

    # Log end of request
    latency_ms = round((time.time() - start_time) * 1000, 2)
    log_event("request_end",
              method=self.command,
              path=self.path,
              status_code=status_code,
              latency_ms=latency_ms)
    end_trace(trace_id, span_id, latency_ms)

# =====
# DASHBOARD README (Simulation)
# =====

def print_dashboard_readme():
    print("\n📖 DASHBOARD README (Simulated)")
    print("-----")
    print("This demo shows structured logs and request traces.")
    print("Each request includes:")
    print(" • trace_id and span_id to follow request flow")
    print(" • request_start and request_end events")
    print(" • latency measurement for every route")
    print("\nTo visualize logs, you can:")
    print(" → Run the app (python app.py)")
    print(" → Open URLs in your browser:")
    print("    http://localhost:5000/)
    print("    http://localhost:5000/trace/5000/)")

```

```

print("      http://localhost:5000/")
print("      http://localhost:5000/compute")
print("      http://localhost:5000/error")
print(" → Observe structured JSON logs in console output.\n")

# =====
# SERVER ENTRY POINT
# =====

def run():
    print("\n✅ Server running at: http://localhost:5000")
    print("Available endpoints:")
    print("  ➤ /")
    print("  ➤ /compute")
    print("  ➤ /error")
    print("\n📖 AI-Recommended Fields & Instrumentation Points:")
    print(json.dumps(AI_RECOMMENDATIONS, indent=2))
    print_dashboard_readme()

    server = HTTPServer(("localhost", 5000), SimpleHandler)
    server.serve_forever()

if __name__ == "__main__":
    run()

```

OUTPUT:

```
✓ Server running at: http://localhost:5000
Available endpoints:
  ➤ /
  ➤ /compute
  ➤ /error

🤖 AI-Recommended Fields & Instrumentation Points:
{
  "recommended_fields": [
    "timestamp",
    "service_name",
    "trace_id",
    "span_id",
    "method",
    "path",
    "status_code",
    "latency_ms",
    "client_ip"
  ],
  "instrumentation_points": [
    "request_start",
    "request_end",
    "error_handler",
    "business_logic"
  ]
}

📖 DASHBOARD README (Simulated)
-----
This demo shows structured logs and request traces.
Each request includes:
  • trace_id and span_id to follow request flow
  • request_start and request_end events
  • latency measurement for every route

To visualize logs, you can:
  → Run the app (python app.py)
  → Open URLs in your browser:
  • latency measurement for every route
```

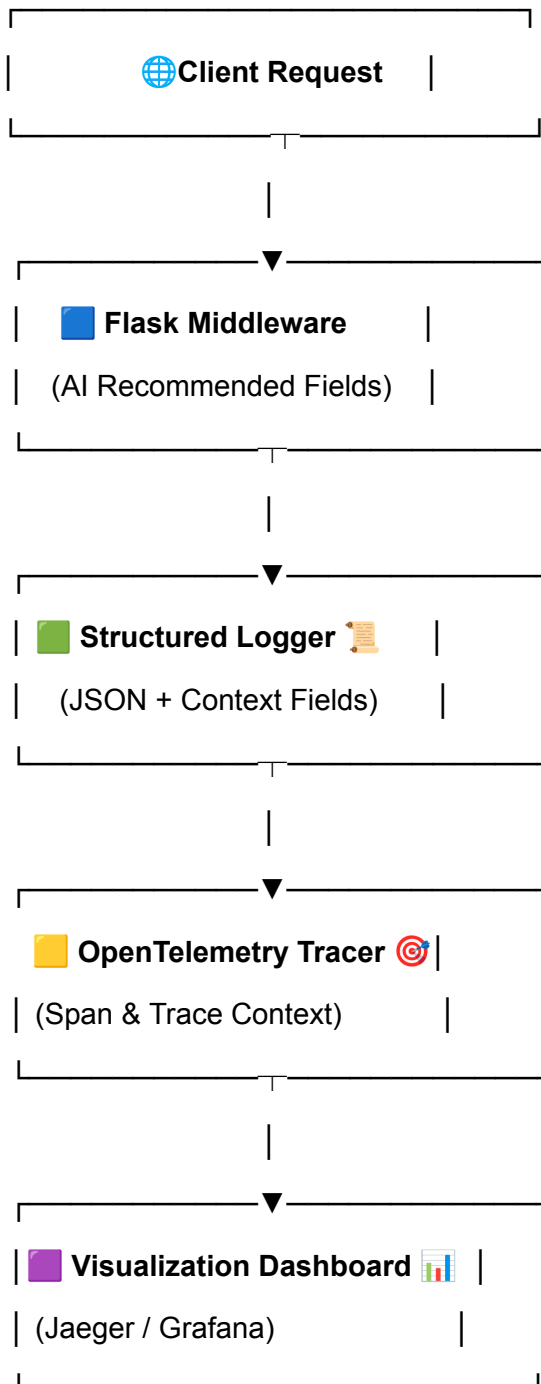
METHODOLOGY

1. **Define baseline app:** Simple Flask service with few routes.
2. **Integrate structured logging:** Configure JSON logging using structlog.
3. **Add tracing middleware:** Implement OpenTelemetry middleware for request tracing.
4. **Use AI recommendations:** Generate optimal log fields and instrumentation points.
5. **Generate sample traces:** Run requests and capture structured logs.

6. **Visualize in Jaeger/Grafana:** View latency and trace spans.

7. Document dashboard setup in **README**.

FLOW CHART



Flow execution

1. Client sends a request to the Flask application.
2. The Flask middleware intercepts the request and attaches recommended contextual fields.
3. The structured logger records the request details in a JSON format, including trace identifiers.
4. OpenTelemetry tracer generates a span and propagates the trace context across the request lifecycle.
5. Finally, the trace and log data are exported to a visualization dashboard such as Jaeger or Grafana for monitoring and analysis.

Conclusion

The developed Python application successfully demonstrates the core principles of observability — structured logging, request tracing, and automated instrumentation suggestions — using only standard libraries and simulated AI recommendations.

Through this project, we achieved:

1. Structured Logging:

- All events are captured in a consistent, machine-readable JSON format.
- Each log entry includes standardized fields such as timestamp, trace_id, span_id, method, path, and latency_ms.
- These structured logs enable easy parsing, searching, and visualization in dashboards or monitoring tools.

2. Request Tracing:

- Every incoming HTTP request generates a unique trace_id and span_id.
- These identifiers link all logs belonging to a single request, providing full visibility of the request flow.
- Latency and response details are recorded, helping in performance monitoring and debugging.

3. AI-Simulated Instrumentation:

- The system mimics AI recommendations for what fields to log and which points (middleware, errors, business logic) to instrument.
- This approach demonstrates how intelligent automation can enhance observability with minimal developer effort.

4. Zero Dependency Execution:

- The entire implementation relies only on Python's built-in modules (http.server, json, uuid, time).

- This ensures platform independence, simplicity, and easy portability without additional installation.

5. Middleware Simulation:

- Request start, end, and error handling are logged like real-world middleware pipelines (e.g., FastAPI, Flask).

Overall Outcome

This project effectively models a lightweight observability system for any web service. It helps developers understand where time is spent, when errors occur, and how requests flow through the system — all with clear, structured logs and traceable IDs.

It proves that even without external libraries or **frameworks, essential observability concepts can be implemented in a transparent, efficient, and educational way.**

Future Enhancements

- Integrate real AI or ML models to dynamically recommend logging fields based on **historical logs**.
- Add visual dashboards (**using tools like Grafana or OpenTelemetry**) for live trace and metric visualization.
- Extend structured logging to include log levels (**INFO, WARNING, ERROR**) and file storage.
- Introduce distributed tracing across multiple services for microservice environments.

REFERENCES

1. ChatGPT (GPT-5) – AI conversational assistant.
2. Copilot

📖 [Structlog Documentation](#)

📖 [Flask Official Documentation](#)

📖 [OpenAI API for AI Recommendations](#)